1. Problem Title & Link

Title: Intersection of Two Arrays

2. Problem Statement (Short Summary)

Given two integer arrays nums1 and nums2, return an array containing the **unique common elements** present in both arrays.

The result can be returned in any order.

3. Examples

Example 1

Input:

nums1 = [1,2,2,1]

nums2 = [2,2]

Output:

[2]

Explanation:

2 appears in both arrays, but only one 2 is returned because duplicates are not allowed.

Example 2

Input:

nums1 = [4,9,5]

nums2 = [9,4,9,8,4]

Output:

[4,9]

Example 3

Input:

nums1=[1,2,3]

nums2=[4,5,6]

Output:

[]

4. Constraints

- 1 <= nums1.length, nums2.length <= 1000
- 0 <= nums[i] <= 1000
- Output must contain unique elements only

5. Core Concept (Pattern / Topic)

HashSet / Hashing

6. Thought Process

Brute Force

Compare every element in nums1 with every element in nums2.

for each i:

  for each j:

    if nums1[i]==nums2[j]

Time Complexity:

O(n × m)

This becomes slow for larger arrays.

Optimized Thinking

Use a HashSet.

Steps:

1. Store all elements of nums1 in a set.
2. Traverse nums2.
3. If element exists in set → add to result set.
4. Convert result set to array.

Reason:

- Set lookup → O(1)
- Duplicate removal happens automatically.

7. Visual Diagram

nums1 = [1,2,2,1]

↓

Set1 = {1,2}

nums2 = [2,2]

↓

Check:

2 ∈ Set1 → add to result

Result = {2}

8. Pseudocode

function intersection(nums1, nums2):

```
create set1
create resultSet

insert nums1 into set1

for each num in nums2:
    if num in set1:
        add num to resultSet

return resultSet as array
```

9. Code Implementation

Python

```
class Solution:
  def intersection(self, nums1, nums2):
    set1 = set(nums1)
    result = set()

    for num in nums2:
      if num in set1:
        result.add(num)

    return list(result)
```

Java

```
import java.util.*;

class Solution {

  public int[] intersection(int[] nums1, int[] nums2) {

    HashSet<Integer> set1 = new HashSet<>();
    HashSet<Integer> result = new HashSet<>();

    for(int num : nums1){
      set1.add(num);
    }
```

```
        for(int num : nums2){
            if(set1.contains(num)){
                result.add(num);
            }
        }

        int[] ans = new int[result.size()];
        int index = 0;

        for(int num : result){
            ans[index++] = num;
        }

        return ans;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n + m)

- n → size of first array
- m → size of second array

Space Complexity

O(n)`

Because of the HashSet.

11. Common Mistakes / Edge Cases

- Returning duplicates
- Using list instead of set
- Forgetting empty arrays
- Returning set directly in Java

12. Dry Run

Step	num	set1	result
1	-	{1,2}	{}

2	2	{1,2}	{2}
3	2	{1,2}	{2}

Final:

[2]

13. Real-Life Use Cases

- Finding common students between two class lists
- Mutual friends in social networks
- Common purchased products
- Common keywords between documents

14. Common Traps

- Adding directly into array instead of set
- Forgetting uniqueness requirement
- Confusing with "Intersection of Two Arrays II" (duplicates allowed)

15. Builds To (Related Problems)

- Intersection of Two Arrays II
- Contains Duplicate
- Two Sum

16. Alternate Approaches

Approach	Time	Space	Notes
Brute Force	O(n×m)	O(1)	Slow
Sorting + Two Pointers	O(nlogn+mlogm)	O(1)	Good alternative
HashSet	O(n+m)	O(n)	Optimal

17. Why This Solution Works

HashSet provides constant-time lookup and automatically removes duplicates. We only check whether an element exists in the other array and store unique matches.

18. Follow-up Questions

1. How to return duplicates also?
2. Can we solve using two pointers?

3. Can we do this with O(1) extra space?
4. How would this work for streaming data?
5.